Day 171 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 25, Day 4 reference

Watch: L53: Early stopping: preventing overfitting with validation monitoring (IIT Madras) | Early Stopping in Keras to Prevent Overfitting (Jeff Heaton)

`day171_early_stopping.py`, segment by segment, in the order `main()` uses things. Today's new parts are copy_net, EarlyStopper, stop_at, gl_stop_at, row_losses, train_epoch and train_es, copied unchanged into day171_exercise.py; Day 170's exercise (and Day 126's network through it) and Day 161's Adult data are imported. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: net = a list of (W, b) layers · acts = each layer's activation · curve = the stop set's loss after each epoch · p = patience · s, b = the stopping epoch and the best epoch (counted from 0) · T = recorded runs per training size · A, B = two halves of the stop set

Segment 0 — Setup

Imports and constants

CODE

```
sys.path.insert(0, HERE)
sys.path.insert(0, os.path.join(ROOT, "week-24-model-evaluation-statistics"))
with contextlib.redirect_stdout(io.StringIO()):
    import day170_exercise as d170
    import day161_exercise as d161
d126 = d170.d126
MAX_EPOCHS = 100
LR = 1e-3
SIZES = (2000, 18000)
STOP_ROWS = 6000
SEEDS = 10
PATIENCES = (1, 2, 5, 10, 20)
GL_ALPHAS = (1.0, 2.0, 5.0)
HALVES = (100, 500, 3000)
SPLITS = 100
```

- `sys.path.insert(0, HERE)` — this week's folder first, so Day 170's exercise imports by name.
- `d126 = d170.d126` — a module imported inside another is reachable as its attribute; no second import needed.
- `SIZES = (2000, 18000)` — a tuple: a fixed list of settings the loops run over.
- `GL_ALPHAS = (1.0, 2.0, 5.0)` — percent thresholds for Prechelt's rule.

Segment 1 — The parts

copy_net and EarlyStopper

MATH

stop at the first t with $t - t^* \geq p$, t^* = the epoch of the best score so far

CODE

```
def copy_net(net):
    return [(W.copy(), b.copy()) for W, b in net]
class EarlyStopper:
    def __init__(self, patience=5, min_delta=0.0):
        self.patience, self.min_delta = patience, min_delta
        self.best, self.best_epoch, self.best_net, self.wait = np.inf, -1, None, 0
    def update(self, epoch, score, net=None):
        if score < self.best - self.min_delta:
            self.best, self.best_epoch, self.wait = score, epoch, 0
            self.best_net = copy_net(net) if net is not None else None
        else:
            self.wait += 1
        return self.wait >= self.patience
```

- `W.copy()` — a new array; without it the saved 'best' weights would keep changing as training goes on.
- `self.best, self.best_epoch, ... = np.inf, -1, None, 0` — four attributes set at once; best starts at infinity so the first score always improves it.
- `score < self.best - self.min_delta` — an improvement must beat the best by more than min_delta.
- `copy_net(net) if net is not None else None` — a conditional expression: replaying a curve passes no network.
- `return self.wait >= self.patience` — a comparison is already True or False; no if needed.

stop_at and gl_stop_at: rules on a recorded curve

MATH

$GL(t) = 100\,(E(t) / \min_{s \leq t} E(s) - 1)$

CODE

```
def stop_at(curve, patience, min_delta=0.0):
    es = EarlyStopper(patience, min_delta)
    for e, v in enumerate(curve):
        if es.update(e, v):
            return e, es.best_epoch
    return len(curve) - 1, es.best_epoch
def gl_stop_at(curve, alpha):
    best = np.minimum.accumulate(np.asarray(curve, float))
    gl = 100 * (np.asarray(curve, float) / best - 1)
    hit = np.flatnonzero(gl > alpha)
    e = int(hit[0]) if len(hit) else len(curve) - 1
    return e, int(np.argmin(curve[:e + 1]))
```

- `es = EarlyStopper(patience, min_delta)` — the replay uses the same class as real training, so the rule cannot differ.
- `return e, es.best_epoch` — returning from inside the loop ends the function at once.
- `np.minimum.accumulate(...)` — the running minimum: element t is the smallest of elements 0 to t.
- `np.flatnonzero(gl > alpha)` — the positions where the condition holds.

- `int(hit[0]) if len(hit) else len(curve) - 1` — the first such epoch, or the last epoch if the rule never fires.
- `curve[:e + 1]` — the curve up to and including the stopping epoch: the rule cannot see the future.

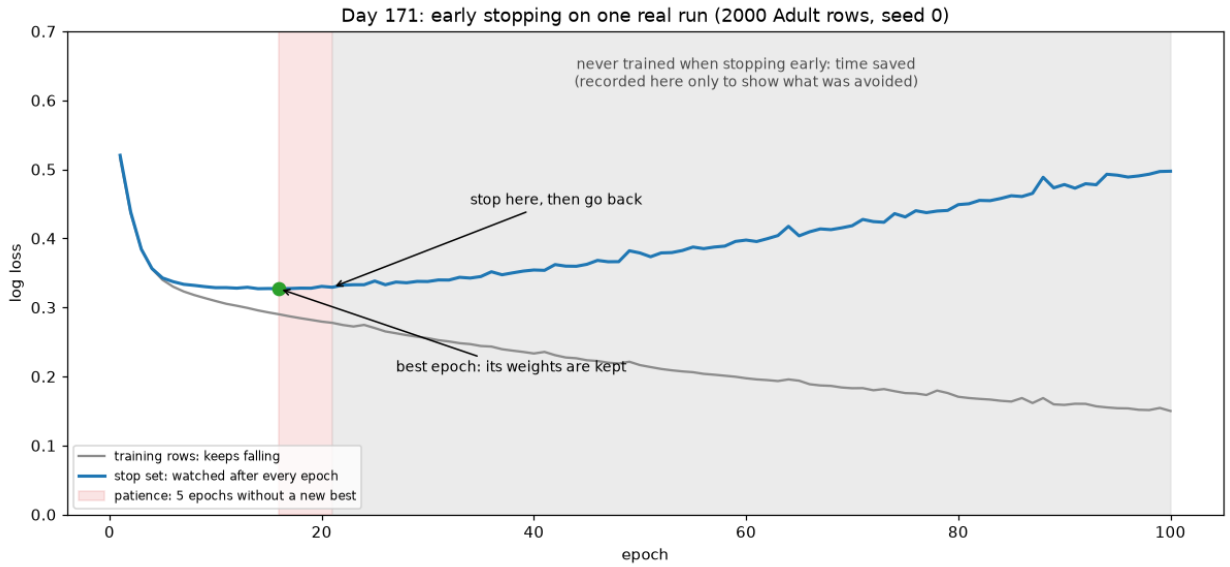
row_losses, train_epoch and train_es

CODE

```
def row_losses(net, acts, X, y):
    z = d126.forward(net, X, acts)[-1][:, 0].astype(float)
    return np.logaddexp(0, z) - y * z
def train_epoch(net, opt, acts, Xa, ya, rng, batch=128):
    o = rng.permutation(len(Xa))
    for s in range(0, len(Xa), batch):
        A = d126.forward(net, Xa[o[s:s + batch]], acts)
        opt.step(net, d126.backward(net, A, acts, d170.bce_logits(A[-1], ya[o[s:s +
            batch]])[1])[0])
def train_es(Xa, ya, Xs, ys, opt_cls, lr, seed, stopper, max_epochs=MAX_EPOCHS, batch=128):
    sizes = [Xa.shape[1], *d170.HIDDEN, 1]; acts = ["relu"] * len(d170.HIDDEN) + ["linear"]
    net = d126.init_net(sizes, seed); opt = opt_cls(net, lr); rng = np.random.RandomState(seed +
        1)
    curve = []
    for e in range(max_epochs):
        train_epoch(net, opt, acts, Xa, ya, rng, batch)
        curve.append(row_losses(net, acts, Xs, ys).mean())
        if stopper.update(e, curve[-1], net):
            break
    return stopper.best_net, acts, np.array(curve)
```

- `[:, 0].astype(float)` — one logit per row, in float64, so every caller averages it the same way.
- `np.logaddexp(0, z) - y * z` — each row's log loss on its logit, without overflow (Day 170).
- `rng.permutation(len(Xa))` — a new order each epoch, from the run's own generator.
- `stopper.update(e, curve[-1], net)` — curve[-1] is the score just appended; the network is passed so the stopper can copy it.
- `return stopper.best_net, acts, np.array(curve)` — the best epoch's weights, not the last; the curve shows how far it went.

SYSTEM DIAGRAM



One run: the best epoch, the patience window, the stop, and the epochs never trained.

Segment 2 — The experiments

trajectory: recording every epoch

CODE

```
def trajectory(D, n, seed, shuffle=False):
    Xa, ya = D["X"][D["train"][:n]], D["y"][D["train"][:n]]
    if shuffle:
        ya = ya[np.random.RandomState(99 + seed).permutation(n)]
    sizes = [Xa.shape[1], *d170.HIDDEN, 1]; acts = ["relu"] * len(d170.HIDDEN) + ["linear"]
    net = d126.init_net(sizes, seed); opt = d126.Adam(net, LR); rng = np.random.RandomState(seed +
        1)
    tr, st, rows, fr = [], [], [], []
    t0 = time.time()
    for e in range(MAX_EPOCHS):
        train_epoch(net, opt, acts, Xa, ya, rng)
        tr.append(row_losses(net, acts, Xa, ya).mean())
        ls = row_losses(net, acts, D["X"][D["stop"]], D["y"][D["stop"]])
        st.append(ls.mean()); rows.append(ls.astype(np.float32))
        fr.append(row_losses(net, acts, D["X"][D["fresh"]], D["y"][D["fresh"]]).mean())
    return dict(train=np.array(tr), stop=np.array(st), rows=np.array(rows), fresh=np.array(fr),
        sec=time.time() - t0)
```

- `D["X"][D["train"][:n]]` — index arrays: the first n of the training pool's row numbers pick the rows.
- `ya[np.random.RandomState(99 + seed).permutation(n)]` — the control: the same labels in a random order, so they no longer match the rows.
- `tr, st, rows, fr = [], [], [], []` — four empty lists, one per thing recorded.
- `ls.astype(np.float32)` — each stop-set row's loss kept at half the memory, for section 4.
- `dict(train=np.array(tr), ...)` — lists become arrays once, at the end.

rule_rows: every rule on every run

CODE

```
rules = [(f"patience {p}", lambda c, p=p: stop_at(c, p)) for p in PATIENCES]
rules += [(f"GL > {a:g}%", lambda c, a=a: gl_stop_at(c, a)) for a in GL_ALPHAS]
rules += [("no stopping, best", lambda c: (len(c) - 1, int(np.argmin(c)))),
          ("no stopping, last", lambda c: (len(c) - 1, len(c) - 1))]
out = []
for name, rule in rules:
    se = [rule(r["stop"]) for r in R]
    fr = [r["fresh"][b] for r, (s, b) in zip(R, se)]
    rg = [f - r["fresh"].min() for r, f in zip(R, fr)]
    hit = np.mean([b == int(np.argmin(r["stop"])) for r, (s, b) in zip(R, se)])
    out.append((name, np.mean([s + 1 for s, b in se]), np.mean(fr), np.mean(rg), max(rg), hit))
return out
```

- `lambda c, p=p: stop_at(c, p)` — p=p freezes the current p; without it every lambda would use the loop's last p.
- `rules += [...]` — adds the items of another list to this one.
- `for r, (s, b) in zip(R, se)` — unpacks each run and its (stop, best) pair together.
- `np.mean([b == int(np.argmin(r["stop"])) ...])` — the mean of True/False values is the share that are True.

demo_stopset: two halves and a paired control

CODE

```
rng = np.random.RandomState(1000 * k + m)
for _ in range(SPLITS):
    i = rng.permutation(STOP_ROWS)[:2 * m]; a, b = i[:m], i[m:]
    ca, cb = r["rows"][:, a].astype(float).mean(1), r["rows"][:, b].astype(float).mean(1)
    s, e = stop_at(ca, 5)
    rg.append(r["fresh"][e] - r["fresh"].min()); ep.append(s + 1)
    opt.append(cb[e] - ca[e])
    f = int(np.argmin(r["stop"]))
    ctl.append(cb[f] - ca[f])
np.array(opt) - np.array(ctl); se = d.std() / np.sqrt(len(d))
```

- `rng.permutation(STOP_ROWS)[:2 * m]` — 2m distinct stop-set rows; the first m are A, the rest B.
- `r["rows"][:, a].astype(float).mean(1)` — the columns for A's rows, averaged across rows: A's loss after every epoch.
- `opt.append(cb[e] - ca[e])` — B's score minus A's at the epoch A chose.
- `d = np.array(opt) - np.array(ctl)` — paired: each draw's optimism minus its own control, which cancels the luck of the split.
- `d.std() / np.sqrt(len(d))` — the standard error of the mean difference.

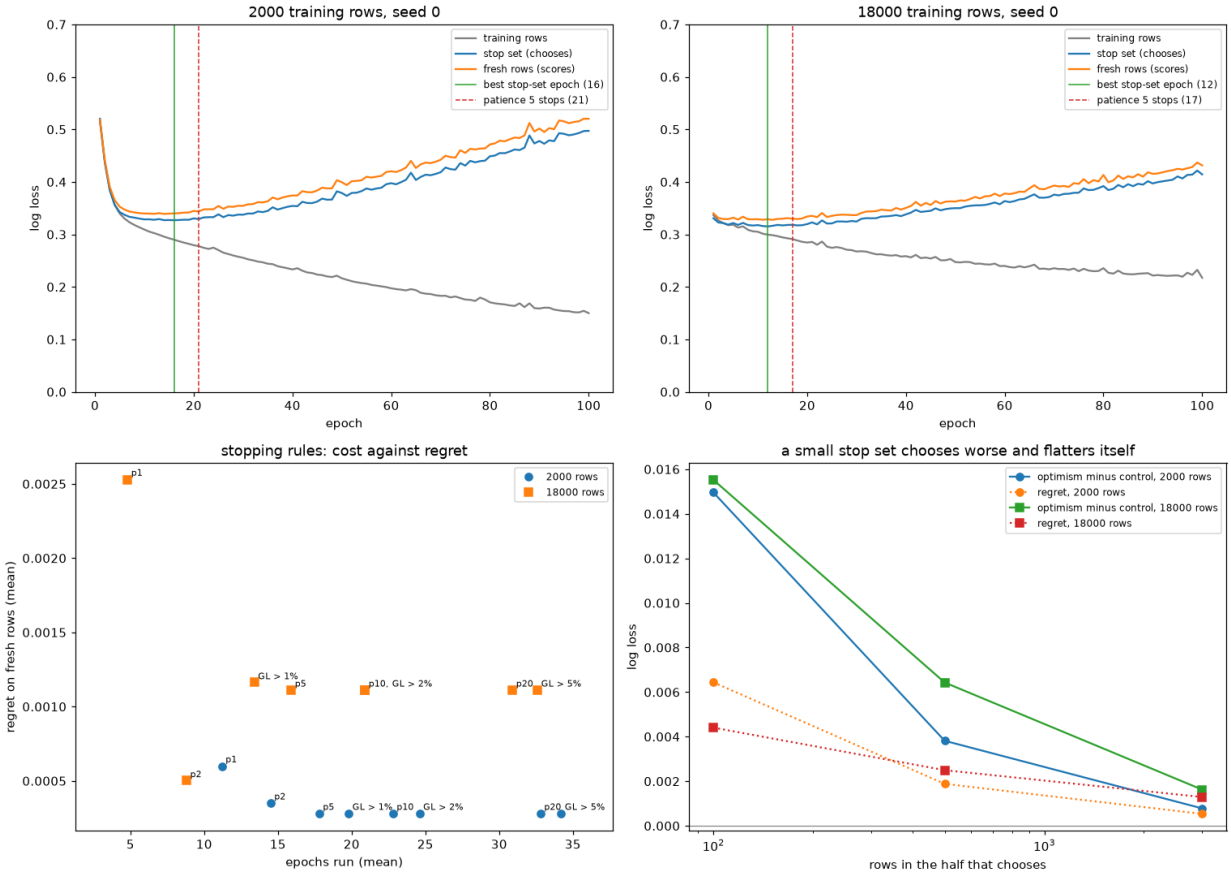
demo_online: checking the replay

CODE

```
net, acts, curve = train_es(Xa, ya, Xs, ys, d126.Adam, LR, 0, EarlyStopper(5))
sec = time.time() - t0
t0 = time.time(); train_es(Xa, ya, Xs, ys, d126.Adam, LR, 0, EarlyStopper(MAX_EPOCHS + 1))
full = time.time() - t0
fr = row_losses(net, acts, D["X"][D["fresh"]], D["y"][D["fresh"]]).mean()
s, b = stop_at(T[n][0]["stop"], 5)
same = np.array_equal(curve, T[n][0]["stop"][:len(curve)])
```

- `EarlyStopper(5)` — a fresh stopper for each run; it holds that run's state.
- `EarlyStopper(MAX_EPOCHS + 1)` — a patience longer than the run: the same loop, never stopping, for a fair timing.
- `np.array_equal(curve, ...[:len(curve)])` — True only if the shapes and every value match exactly.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `x.copy()` — a new array with the same values.
- `np.minimum.accumulate(a)` — the running minimum.
- `np.flatnonzero(mask)` — the positions where a condition holds.
- `lambda c, p=p: ...` — a default argument freezes a loop variable inside a lambda.
- `a += [...]` — extend a list in place.
- `np.array_equal(a, b)` — exact equality of shape and values.
- `mean of booleans` — the share that are True.

End of Day 171 syntax reference. For the experiments and what they showed, see the Day 171 complete guide.